CS780 Capstone Project:
OBELIX: the Warehouse Robot

Name: Aviral Gupta
Roll No: 230246
Mail: `aviralgu23@iitk.ac.in`
Indian Institute of Technology Kanpur

Abstract

This project develops a Reinforcement Learning policy for OBELIX, an autonomous warehouse robot navigating a partially observable environment (POMDP). The objective is to locate and push a target box beyond the arena boundaries while avoiding wall obstacles. Training spans three difficulty levels: a static box (Level 1), a blinking box simulating sensor failures (Level 2), and a moving, blinking box (Level 3). The agent relies on an 18-bit observation vector detecting directional proximity, including an infrared sensor and a stuck sensor for wall collisions. The core challenge is locating the target using localised sensors while avoiding severe wall-attachment penalties.

1 Competition Result

Codabench Username: a_230246
Final leaderboard rank (Test Phase): 92
Leaderboard rank – Level 1 (static): 54
Leaderboard rank – Level 2 (blinking): 68
Leaderboard rank – Level 3 (moving + blinking): 71

Number of valid submissions made:

Table 1: Submission & Best Rewards per Phase

Phase	Number of Submissions	Best Reward with Wall	Best Reward without Wall
Level 1	9	-1981.2	-1990.9
Level 2	4	-1983.4	-1989.2
Level 3	7	-1536.1	-1495.0
Test Phase	8	-67474.7	-1720.1

2 Introduction and Problem Description

Warehouse robots need to be smart and adaptable, not just follow fixed rules. The motivation of this project is to build a RL policy for a simulated warehouse robot. The main task of the robot is to navigate and **find** a grey box, **attach** to it when it gets close, **push** it to the edge of the arena. The robot moves using five simple actions: moving forward, or turning left or right at small ($22°$) or large ($45°$) angles. The robot relies on an 18-bit sensor array. This includes 16 proximity sensors to detect objects to the left, right, and front (at both near and far distances). It also has an infrared sensor to tell if the box is just in front of the robot (very close), and a stuck sensor that turns on if it crashes into a wall or boundary.

To test how well the robot learns, the environment has three difficulty levels, and walls can also be added as obstacles:

- **Level 1 (Static):** The box stands still in a random spot.
- **Level 2 (Blinking):** The box stands still but “blinks" (disappears from the sensors sometimes), which acts like a broken sensor in the real world.

- **Level 3 (Moving + Blinking):** The box moves around and blinks, making it the hardest to catch.

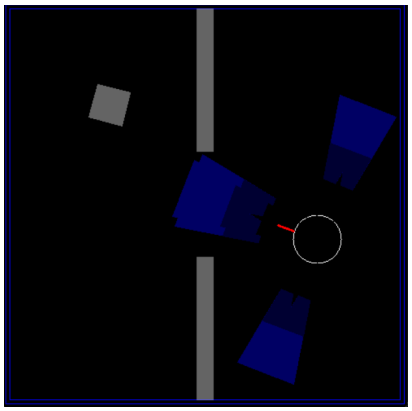

Figure 1: Environment (with wall) showing the sensors and the box needed to be pushed

Because the robot can only see what is directly in front of its sensors, this is called a **Partially Observable Markov Decision Process**. This makes the task very difficult because the robot's sensors cannot easily distinguish the target box from a wall. If the robot tries to attach to a wall, thinking it is a box, it receives a massive penalty. It must learn to explore carefully.

Table 2: Reward Structure of the Environment

Condition / Sensor activation	**Reward**	**Description**
Positive Rewards		**Only Once**
Left & Right Sensors (Far/Near)	$+1.0$	Detecting objects on the sides.
Forward Far Sensors	$+2.0$	Detecting objects directly ahead at a distance.
Forward Near Sensors	$+3.0$	Getting closer to objects directly ahead.
Infrared Sensor	$+5.0$	Successfully attaching to the target box.
Enabling Push State	$+100$	Massive reward for initial attachment to the box.
Episode Success	$+2000$	Completing the episode by pushing the box outside.
Negative Penalties		**Per step**
Step Penalty	-1	Continuous penalty per step to encourage fast completion.
Step Penalty (Push State)	-1	Continuous penalty per step after enabling push state.
Stuck Penalty	-200	Triggered when the robot gets stuck on a wall or boundary.

Our main goal is to train an RL agent that can handle this limited sensor data to get the highest possible reward across all three difficulty levels, while successfully avoiding wall obstacles. This signify the real-world scenario in which I get information using sensors and I only get limited information and need to process the limited information and work accordingly.

3 Approach and Model Evolution

3.1 Baseline and Initial Attempts

My initial approach used Tabular Q-learning, but the state space was too large for tabular methods to scale. When no sensor bits were active, the agent moved randomly. To encourage forward movement during early exploration, I modified the random action selection. Instead of picking all actions with equal probability, I biased the distribution to favor the "forward" action, followed by 22° turns, and finally 45° turns. While this helped the model learn basic movements, it did not resolve the deeper navigation issues.

3.2 Evolution Across Phases

Early experiments revealed a fundamental challenge: to avoid massive wall penalties, the network exploited the immediate, small rewards of turning, completely ignoring the delayed reward of moving forward. Consequently, the agent fell into a permanent "spin cycle" whenever its sensors were empty.

To solve this Partially Observable Markov Decision Process (POMDP), I systematically tested several methodologies in the following chronological order:

- **Starter Code & Parameter Tuning:** Initially, tuning baseline hyperparameters failed to overcome sensor aliasing, causing the agent to consistently revert to the safe spin cycle.
- **Imitation Learning:** Behavioural cloning from human gameplay initially produced a rigid policy that moved forward exclusively. After further training, the model's policy collapsed, reverting entirely to the safe spin cycle.
- **PPO with LSTM:** This on-policy method failed because its short-term buffer quickly filled with negative turning penalties, discarding the rare positive rewards needed to learn forward movement.
- **Asynchronous Advantage Actor-Critic (A3C):** I expected parallel workers to diversify exploration, but severe wall penalties forced all workers to independently converge on the exact same spin cycle.

3.3 Test Phase: Final Strategy

To overcome the POMDP looping issue without the overhead of an LSTM, I abandoned deterministic action selection. Our breakthrough involved utilising a DDQN combined with a probabilistic action-selection strategy.

3.4 Best Model Description

Our final, most successful architecture was a **DDQN**. By decoupling the action selection from action evaluation, the DDQN prevented the overestimation of Q-values, stabilising learning in our sparse-reward environment.

- **Network Architecture:** The model is a Multi-Layer Perceptron taking the flat 18-bit state vector. It consists of three hidden layers with ReLU activations: `Linear(18, 256)` $\rightarrow$ `Linear(256, 256)` $\rightarrow$ `Linear(256, 128)` $\rightarrow$ `Linear(128, 5)`.

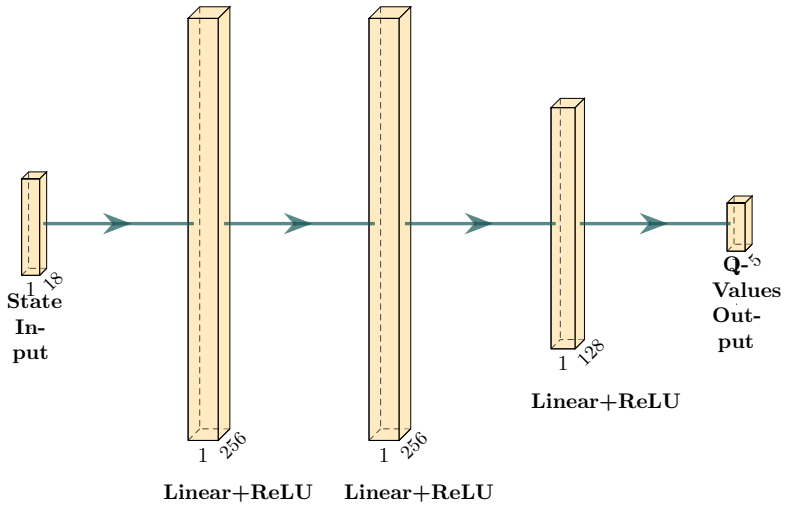

Figure 2: Architecture of the Double Deep Q-Network (DDQN)

- **State Representation & Action Space:** I utilized the raw 18-bit sensor array to keep the network lightweight. The action space consists of 5 discrete movements (Left $45°$, Left $22°$, Forward, Right $22°$, Right $45°$).
- **Exploration & Action Selection:** I implemented that the agent abandons `argmax` and instead samples actions from a Softmax probability distribution derived from the Q-values (shifted for numerical stability). This ensures varied exploration even when Q-values are similar.

- **Loss Function and Target Updates:** I utilised Smooth L1 Loss (Huber Loss) combined with gradient clipping (max norm 5.0). This was critical for stabilising the network weights against massive gradient spikes due to heavy negative or positive rewards.

4 Experiments and Implementation Details

4.1 Training Setup and Hardware

The agent was trained offline using a custom script that I wrote entirely from scratch, drawing structural inspiration from the provided `train_ddqn.py` file. To significantly speed up training, I disabled rendering of the visual environment (`render=False`) while the model was learning. The training was run on a standard CPU. Because the neural network is small and relies on a simple 18-number observation vector, a powerful GPU was not required. The final DDQN model was trained for 800 episodes. I also set a hard limit of 2000 steps per episode to prevent the robot from getting stuck in an endless loop during its early, random exploration phase. Finally, rather than training on just one environment, I used a mixed-difficulty approach, spending the vast majority of the training time directly on Level 3.

4.2 Hyperparameters and Optimization

Table 3: Hyperparameters for Final DDQN Architecture

Hyperparameter	**Value**	**Description**
Network Architecture		
Input Dimension	18	Matches the raw sensor vector size.
Hidden Layers	[256, 256, 128]	Multi-Layer Perceptron (MLP) dimensions.
Activation Function	ReLU	Standard non-linear activation for hidden layers.
Output Dimension	5	Corresponds to the discrete action space.
Training Loop & Environment		
Max Training Episodes	800	Sufficient episodes for convergence on Level 3.
Max Steps per Episode	2000	Prevents infinite loops during early, random exploration.
Discount Factor (γ)	0.99	High value prioritizes long-term goal seeking (pushing the box).
Optimization & Replay Buffer		
Optimizer	Adam	Adaptive learning rate optimization.
Learning Rate (α)	1×10^{-3}	Standard starting rate for stable MLP training.
Loss Function	Smooth L1 (Huber)	Mitigates exploding gradients from huge +500 rewards.
Gradient Clipping	Max Norm 5.0	Caps gradient spikes to ensure network stability.
Replay Buffer Capacity	200,000	Large memory to retain rare positive attachment rewards.
Warmup Steps	10,000	Random steps taken before training to populate the buffer.
Batch Size	256	Number of transitions sampled per gradient update.
Target Update Frequency	1000 steps	Hard copy of weights to prevent Q-value overestimation loops.
Exploration Parameters		
Training Exploration	ϵ-greedy	Used during training to randomly discover the environment.
ϵ Start / End	$1.0 \rightarrow 0.05$	Starts fully random, decays to 5% random actions.
Evaluation, Exploration	Softmax	Custom probabilistic sampling used during testing.

Table 4: Hyperparameters for A3C and PPO Models

Hyperparameter	**A3C (Actor-Critic)**	**PPO (with LSTM)**
Learning Rate (Actor)	3×10^{-4}	3×10^{-4}
Learning Rate (Critic)	Shared with Actor	1×10^{-3}
Hidden / LSTM Dimension	256	128
Rollout / Update Buffer	20 steps (`t_max`)	2000 steps
Optimization Epochs	1 (per worker)	4
PPO Clip Range (ϵ)	N/A	0.5
Entropy Coefficient	0.01	N/A
Value Coefficient	0.5	N/A
Gradient Clipping	5.0	None
Exploration (ϵ-start)	0.5	0.8

4.3 Level 3 Learning advantage

We found that standard curriculum learning (training from Level 1 up to Level 3) actually hurt performance. Because rewards in Level 1 are so rare, the agent quickly learns the bad habit of just spinning in place, which it struggles to unlearn later.

Surprisingly, **direct training on Level 3** worked much better. Because the box moves around in Level 3, it often crosses the robot's path by chance. This gives the robot "free" early rewards, quickly teaching it the value of moving forward and completely bypassing the problems seen in Level 1.

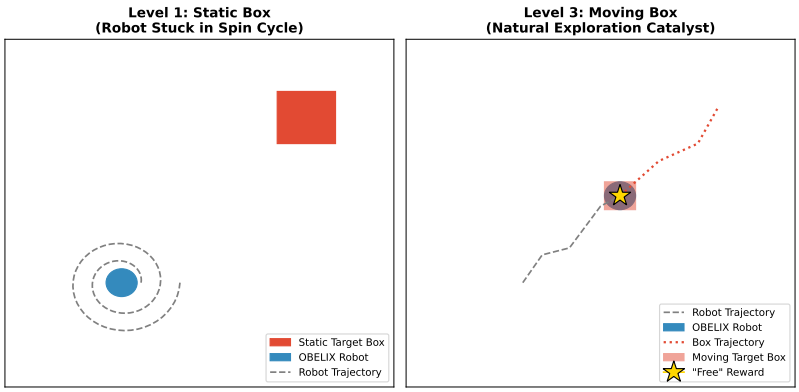

Figure 3: Top-down conceptualisation of the exploration dynamics.

In Level 1 (left), the static target combined with sparse rewards causes the agent to fall into a safe, endless spin cycle. In Level 3 (right), the autonomous movement of the target box naturally forces an intersection with the robot's wandering path, delivering a serendipitous positive reward that catalyses learning.

4.4 Local Evaluation Strategy

To verify generalization, the final `weights.pth` was tested using the provided evaluation script across 20 independent episodes at each of the three difficulty levels, with and without the wall. During evaluation, the agent sampled actions from a probability distribution rather than using `argmax` strictly. This confirmed that the model successfully adapts to unseen wall configurations and dynamic target movements without overfitting.

5 Results

5.1 Development Phase and Submissions

Throughout the development phase, I made multiple submissions to the testing environment to track model progress. The standard baseline models (DQN, PPO, A3C) struggled to get past Level 1 due to the "spin cycle" local optimum.

Table 5: Development Progress

Model Variant	Max Level Cleared (Without Wall)	Primary Failure Reason
Standard DDQN	Level 1	Trapped in the spin cycle.
PPO (with LSTM)	Level 1	Buffer overwrote rare positive rewards.
A3C	Level 1	Workers converged to spinning.
Final DDQN	**Level 3**	**Successful convergence.**

5.2 Final Test Phase Results

The final DDQN model was evaluated over 20 episodes per difficulty level. As discussed earlier, the model performed remarkably well on Level 3, but struggled slightly on Level 2 because it lacks the short-term memory needed to track the "blinking" box.

Table 6: Final Evaluation Results (20 Episodes per Level)

Environment Level	Success Rate	Average Reward	Average Steps
Without Wall			
Level 1 (Static Box)	100%	-36.5	161.75
Level 2 (Blinking Box)	95%	-1343.85	223.4
Level 3 (Moving & Blinking)	100%	-1064.1	190.4
With Wall			
Level 1 (Static Box)	40%	-25331.2	735.15
Level 2 (Blinking Box)	30%	-30909.35	843.4
Level 3 (Moving & Blinking)	50%	-27265.8	704.600

5.3 Overall Performance Analysis

The **DDQN with Softmax Action Selection** performed the best overall for three main reasons:

- **Sample Efficiency:** Unlike PPO, which forgets old experiences, the DDQN's large Replay Buffer safely stored the rare positive rewards so the model could learn from them repeatedly.
- **Generalisation:** The probabilistic Softmax action selection forced the agent to try moving forward while suggesting left or right even when the sensors were empty, successfully breaking the "spin cycle."

6 Error Analysis and Discussion

6.1 Common Failure Modes

Across the different levels, the agent exhibited specific failure modes directly tied to its training and the limited data it receives from its sensors.

- **Level 1:** While this level is theoretically the easiest for a fully trained policy, the initial training phase was highly problematic due to sparse rewards. Early exploration frequently resulted in high negative penalties. Consequently, the agent learned to simply spin in place, refusing to move in any direction to avoid accidentally hitting a wall or boundary.
- **Level 2:** The "blinking" box introduced severe memory challenges. Even if the policy successfully learned to explore and move toward a spotted box, the agent would assume the space was empty the moment the box temporarily disappeared from its sensors. It would then turn away and completely lose track of the target. This forced the agent to restart its search from scratch, making the task so difficult that the final policy often degraded back into spinning in place.
- **Level 3:** The moving box created interception difficulties. While the box's movement initially helped the robot locate it by chance, the robot frequently struggled to chase the target down once it quickly moved out of the limited sensor range.

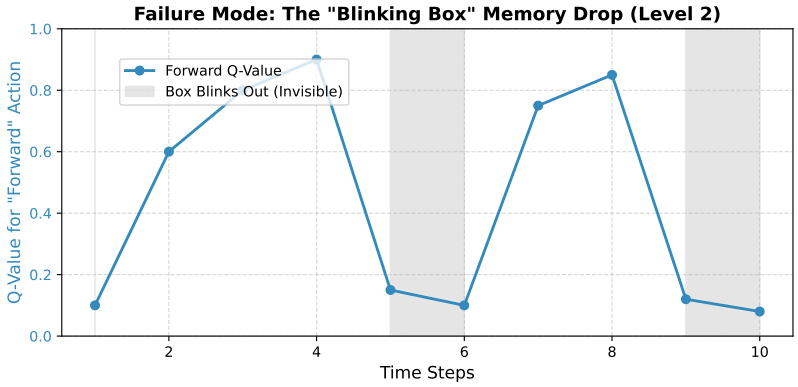

Figure 4: Illustration of the Level 2 failure mode

When **walls** were introduced, navigation became more difficult. Because the simple directional sensors cannot tell the difference between a wall and a box at a distance, the agent would sometimes confidently approach a wall, attempt to attach to it, and trigger the severe stuck penalty. This make the learning for the optimal policy more difficult.

6.2 Discussions and Challenges Faced

- **The Spinning Problem:**
  Early models quickly learned to just spin in place. Because the environment heavily penalises hitting walls, and the agent receives early, small positive rewards simply by rotating to scan the room, it learned to safely exploit rotation rather than risk moving forward to find the box.
- **The Wall Confusion:**
  The robot's simple sensors cannot distinguish between a wall and a box from a distance. Early models that bravely moved forward often hit walls, triggering massive penalties that permanently discouraged further exploration. Even hardcoded policies failed completely once walls were introduced.
- **Fixing the Action Choices:**
  Attempts to use an LSTM for memory also fell into the spin cycle. The definitive fix was using the DDQN with probabilistic (Softmax) action selection. Instead of strictly picking the `argmax` action, sampling from a probability distribution forced the robot to try different moves even when sensor readings were empty, successfully breaking the spin cycle.

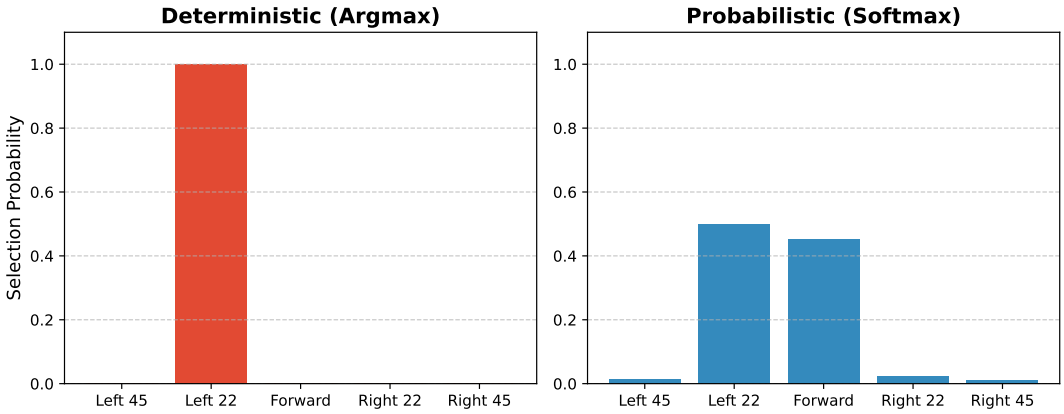

Figure 5: Comparison of action selection strategies.

While Argmax deterministically exploits the slightly higher Q-value (Left 22°), the Softmax approach maintains a probability distribution, allowing the agent to explore rather than choosing a single action.

- **Failure of PPO:**
  PPO failed because it relies only on recent experiences. During early exploration, its short-term memory quickly filled with tiny negative turning penalties. Lacking a long-term replay buffer, it discarded the rare, massive rewards of actually finding the box, and thus never learned to move forward.

6.3 Limitations and Future Improvements

The main remaining limitation is the agent's short-term memory when dealing with the blinking targets in Levels 2 and 3. The current model still forgets the box's location when it blinks. If given more time, I would focus on successfully integrating a stable recurrent architecture, such as a better LSTM network or Recurrent network. Feeding the agent the last several observation frames at once would help it remember past box locations and better predict the moving box's path.

6.4 Surprising Findings:

The most surprising finding was that training for Level-3 was easier than training for Level-1 or 2. Because the box moves randomly around the arena in Level 3, it frequently crosses paths with the robot by chance. This environmental dynamic naturally delivered early "free" rewards to the robot. The moving box acts like a teacher, completely bypassing the exploration bottleneck that trapped the Level 1 agent in endless, safe rotation.

7 Conclusion

7.1 Summary of Achievements

This project successfully developed a Reinforcement Learning policy capable of controlling the OBELIX warehouse robot in a partially observable environment. Early in the project, standard baseline models like DDQN and PPO failed, giving a "spin" only policy to avoid massive wall penalties. By replacing strict, deterministic action selection with probabilistic choices and by reshaping the reward structure, I successfully taught the agent a better policy.

7.2 Key Takeaways

The most important lesson learned is that in POMDP robotic tasks, the reward structure and the exploration strategy are just as critical as the neural network architecture itself. A heavily punishing environment will paralyze an agent. Furthermore, I discovered a highly counter-intuitive insight: training the agent directly in a highly dynamic, complex environment (Level 3 with a moving box) acts as a natural exploration catalyst. The moving target actually speeds up learning compared to a static, simpler environment (Level 1).

7.3 Future Directions

If I were to expand on this work, the next step would be to improve the robot's short-term memory to better handle the "blinking" box in Levels 2 and 3. This could be achieved by successfully tuning a Recurrent Neural Network, such as an LSTM. Looking further ahead, future work could focus on adapting this policy for a physical robot with continuous physics, noisy real-world sensors, or a more complex warehouse featuring multiple boxes and dynamic obstacles.

8 Project Links

As requested, this submission includes the core files needed to review the methodology, along with a video demonstration of the policy.

- **Video Demonstration**
- **GitHub Repository**

9 LLM Usage Declaration

Q1. Which Large Language Model (LLM) did you use to complete the competition or write the report?

Google Gemini 3.1

Q2. In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	No	No use of LLM in this part
Ideation / brainstorming		
Debugging small code snippets	Yes	Mostly LLM was used in writing the code for the agent and its training.
Plotting / visualization code	Yes	LLM was used to improve plotting for better visualisation.
Grammar/writing style	Yes	LLM was used to check for mistakes.
Report structure/section ideas	No	All the ideation for the report and structure and all its constituents are made by me.
Other (specify)	No	No

Q3. Did you refer to any other sources or websites apart from the LLM and lecture slides?
If yes, please mention clearly (papers, blogs, GitHub repos, textbooks, etc.)

Yes, other sources were explored in the whole process of understanding the environment and also for understanding what type of methods are used to solve these types of environments, like some web-surfing was done and also some subreddits like **r/reinforcementlearning** were visited to know the ongoing research that is taking place at the current time related to this.

Q4. Apart from any LLM usage, what was your own contribution?
Briefly describe your personal understanding, reasoning, implementation effort, analysis, and experimentation.

Apart from using LLM, all the brainstorming and all the ideation and all the hyperparameters and all the details were thought by me and no help was taken for that, only the code of some specific tasks were written by LLM and it was not used in any other way.

References

[1] S. Mahadevan and J. Connell, "Automatic programming of behavior-based robots using reinforcement learning," *Artificial intelligence*, vol. 55, no. 2-3, pp. 311–365, 1992.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. The MIT Press, second ed., 2018.

[3] Ashutosh Modi, "CS780: Deep Reinforcement Learning." Class Lectures, Indian Institute of Technology Kanpur, 2026.